Student Name : Alex Carter

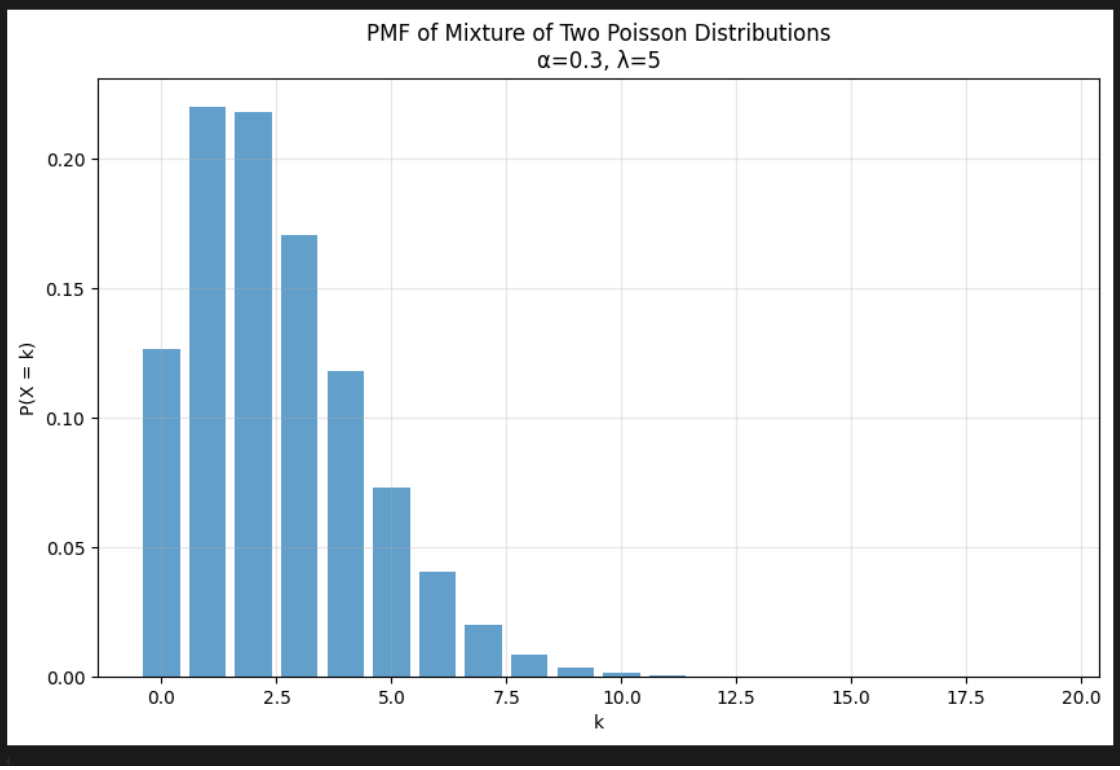
Student Number : 240355490

By submitting my assignment:
I declare that I have personally prepared this assignment. The work is my own, carried out personally by me unless otherwise stated and has not been generated using Artificial Intelligence tools unless specified as a clearly stated approved component of the assessment brief. All sources of information, including quotations, are acknowledged by means of the appropriate citations and references. I declare that this work has not gained credit previously for another module at this or another University. I understand that plagiarism, collusion, copying another student and commissioning (which for the avoidance of doubt includes the use of essay mills and other paid for assessment writing services, as well as unattributed use of work generated by Artificial Intelligence tools) are regarded as offences against the University's Assessment Regulations and may result in formal disciplinary proceedings.
I understand that by submitting this assessment, I declare myself fit to be able to undertake the assessment and accept the outcome of the assessment as valid.

Question 1:

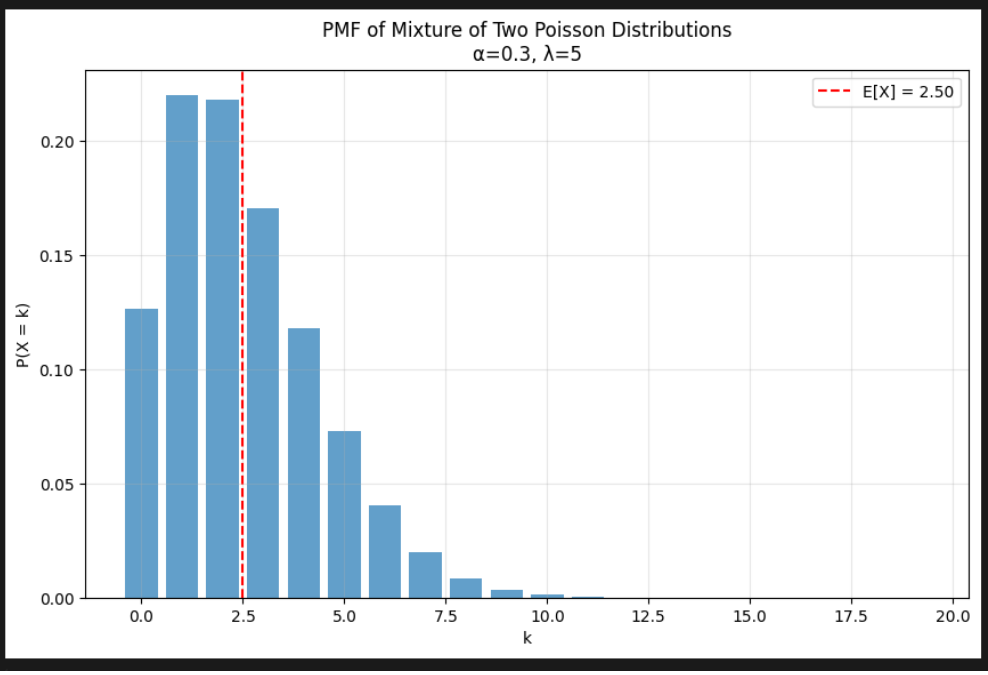
Question 1, a)

```
For k=3, alpha=0.3, lambda=5:
P(X = 3) [numpy] = 0.170648
P(X = 3) [scipy] = 0.170648
```



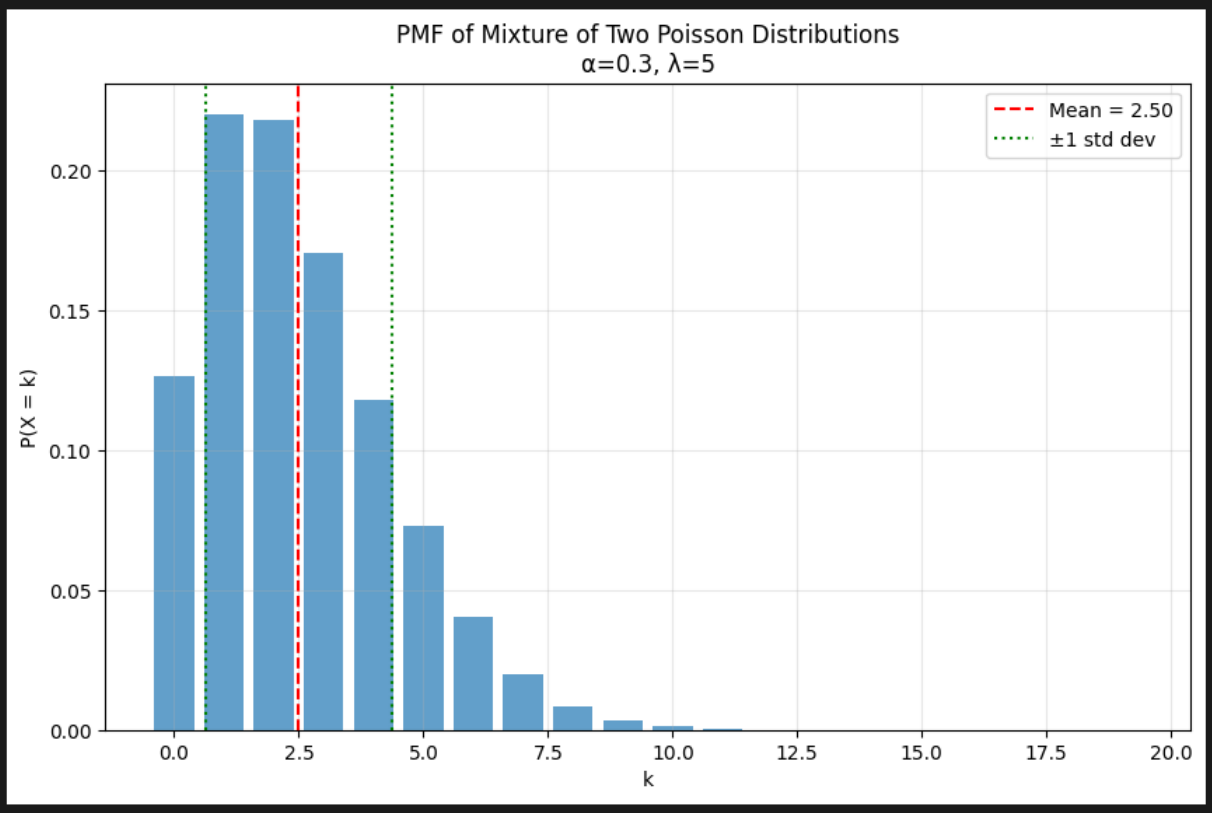
Question 1, b)

```
For alpha=0.3, lambda=5:
Theoretical E[X] = 2.5000
Empirical E[X] = 2.4989
```



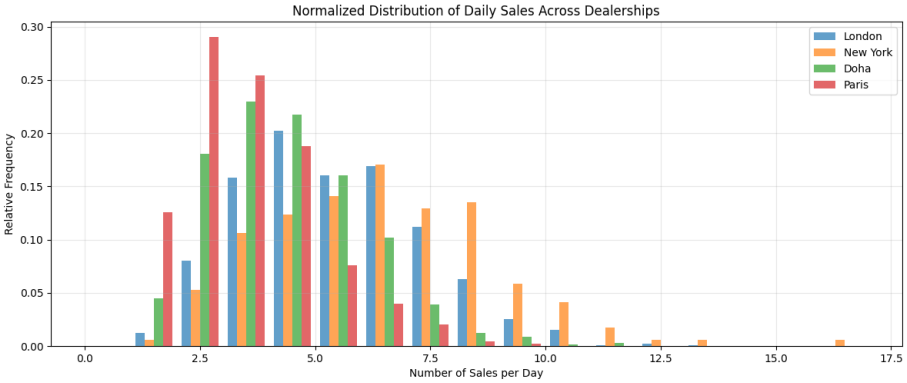
Question 1, c)

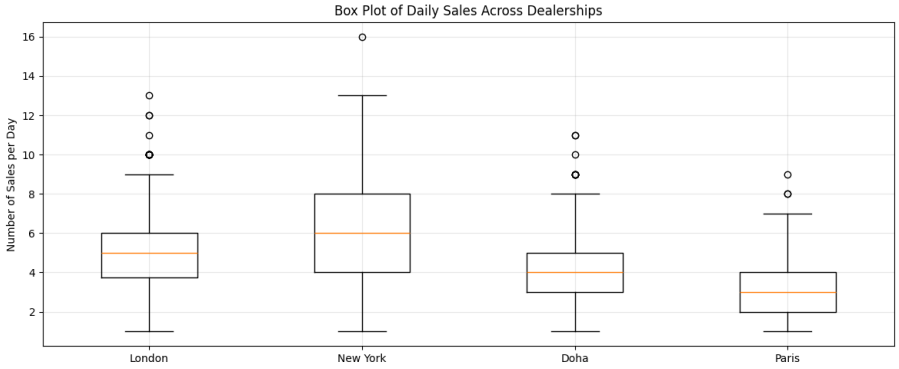
```
For alpha=0.3, lambda=5:
Theoretical Var(X) = 3.5000
Empirical Var(X) = 3.4840
```



Question 2:

Question 2, a)






```
Summary Statistics:
Dealership | Mean | Median | Std Dev | Min | Max | Sample Size
-------------------------------------------------------------
London     | 4.98 | 5.00 | 1.98 | 1 | 13 | 1000
New York   | 6.02 | 6.00 | 2.44 | 1 | 16 | 170
Doha       | 3.88 | 4.00 | 1.70 | 1 | 11 | 666
Paris      | 3.03 | 3.00 | 1.46 | 1 | 9 | 500
```

Comments : Initially, I have applied normalisation to the data so that we can make some comparisons between dealerships; otherwise it may have seemed that London outsold other geographic regions when they are simply the oldest dealership and have collected the most data. After normalisation we can see that New York on average sells the most per day. All dealership sales per day seem to follow a relatively gaussian distribution. Paris seems to have the lowest average sales per day.

Question 2, b)

```
=== Maximum Likelihood Analysis of Sales Patterns ===

London's Sales Pattern:
--------------------------------------------------
Poisson λ: 4.98

Comparison with Other Dealerships:
--------------------------------------------------

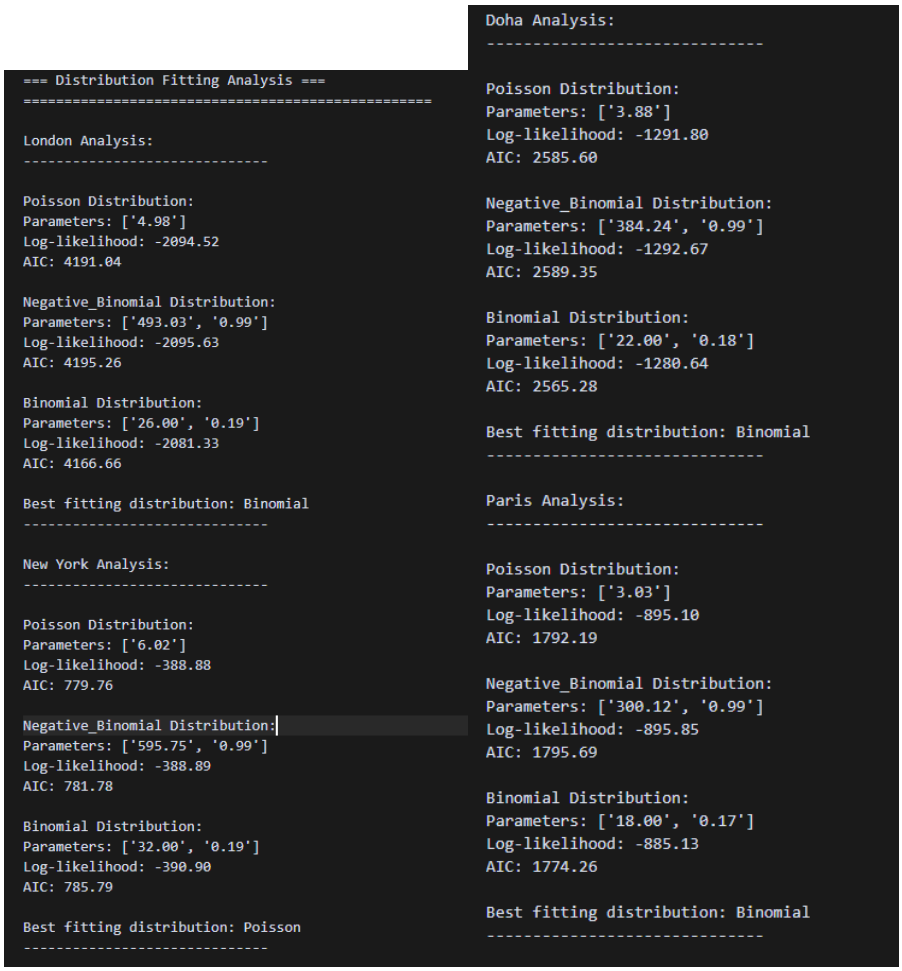
New York:
Poisson λ: 6.02
λ difference: 1.04
KL Divergence: 0.1149

Doha:
Poisson λ: 3.88
λ difference: 1.10
KL Divergence: 0.2605

Paris:
Poisson λ: 3.03
λ difference: 1.95
KL Divergence: 0.9489

==================================================
Conclusion: New York is most similar to London
Based on:
1. KL Divergence: 0.1149
2. λ difference: 1.04
==================================================
```

Question 2, c)



Question 3:

Question 3,

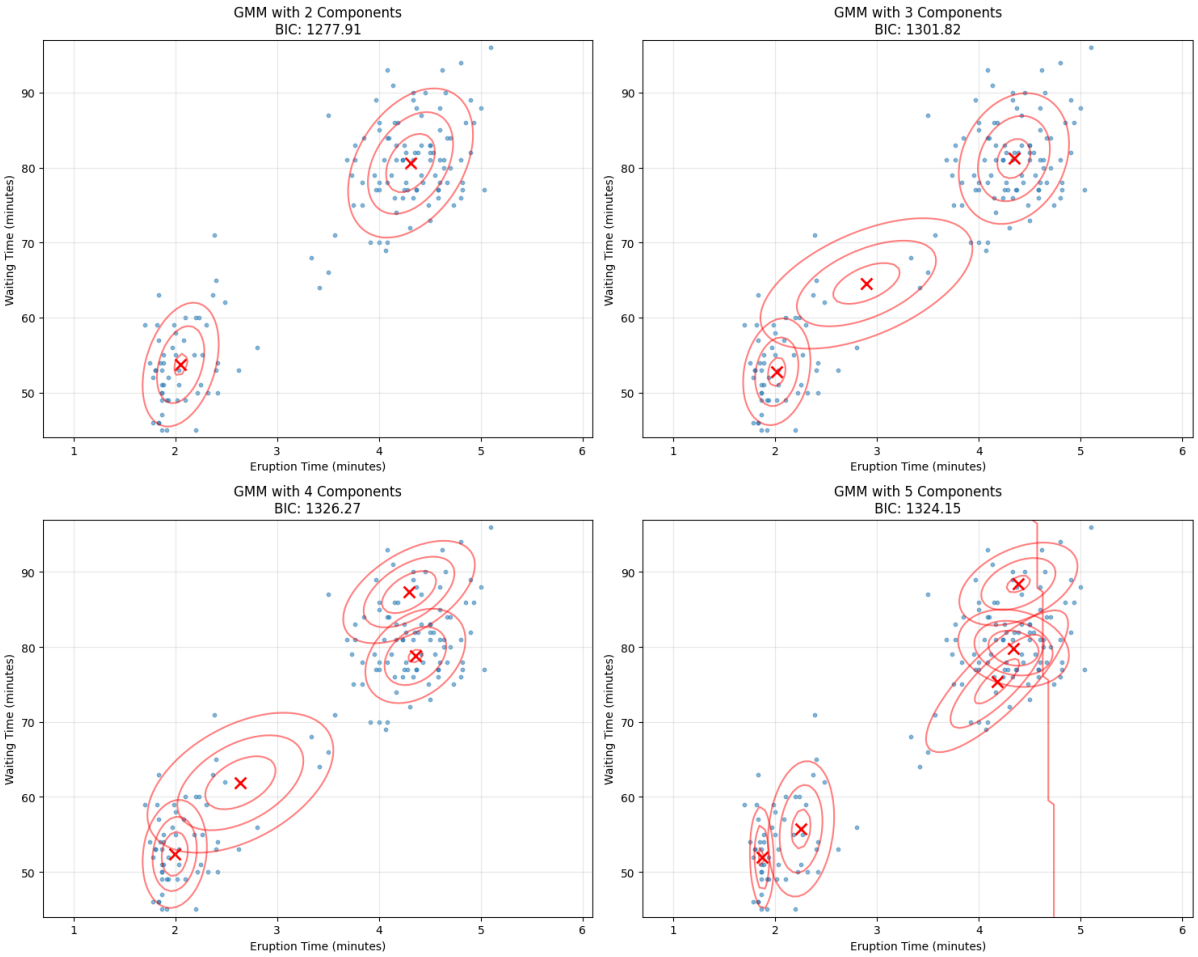
a)

A Gaussian Mixture Model is a probabilistic model that represents data as a combination of multiple Gaussian distributions. In my implementation, I use a 2-component GMM to model the Old Faithful geyser data, where each component represents a different pattern of eruption behaviour. The Expectation-Maximization (EM) algorithm is used to fit the GMM to the data through an iterative process:

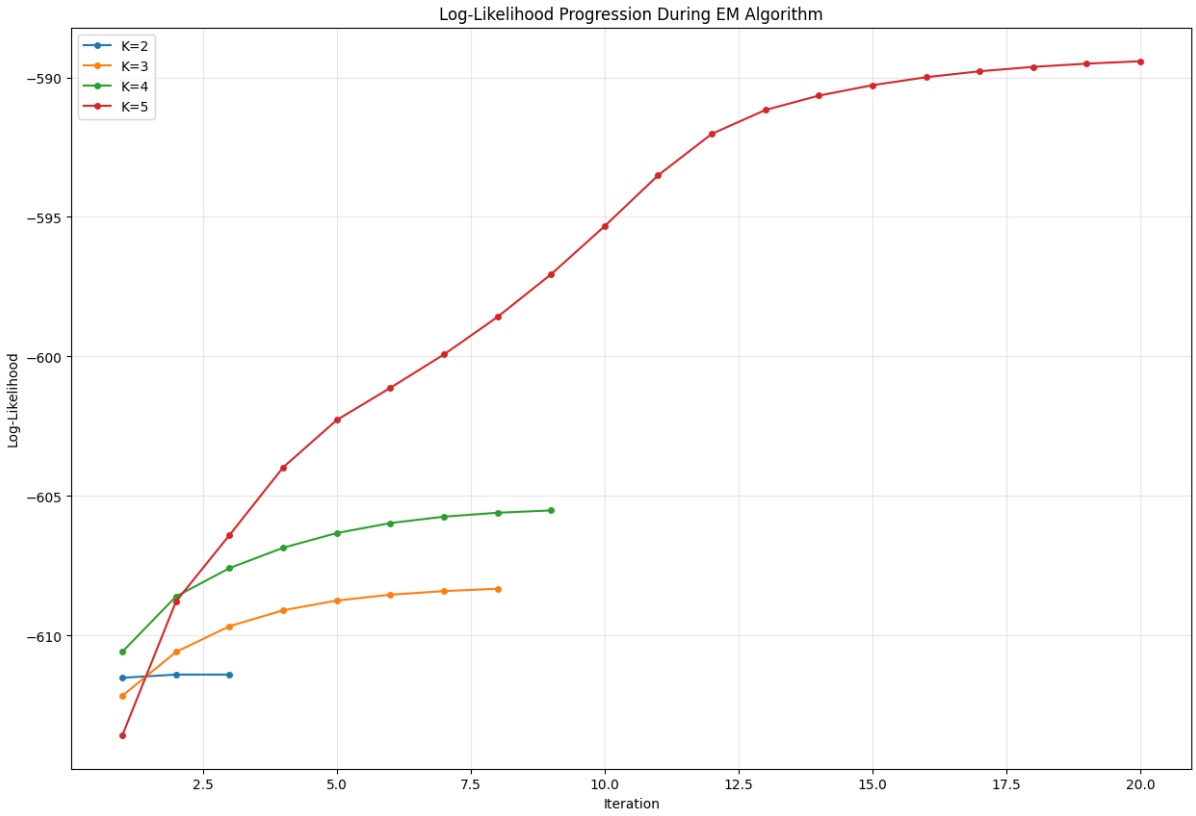
1. **E-step (Expectation):** Calculates the probability that each data point belongs to each Gaussian component
1. **M-step (Maximization):** Updates the parameters (means, covariances, and mixing weights) of each Gaussian component

I chose K=2 components because the data shows two distinct patterns of eruption behaviour. The algorithm iterates until the parameters converge or reach a maximum number of iterations, providing both the cluster assignments and the parameters of the underlying distributions.

Question 3, b)



Question 3, c)



Question 3, d&e)

K=2 best represents the data. This is true for a combination of reasons; primarily because the current literature seems to suggest that the geyser has two main patterns of eruption, which also seems to be what we are observing with our investigation. Furthermore, the log-likelihood progression is much shorter for k=2, this also suggest that this is the most simple pattern to reconstruct, especially compared to k =[3,4,5]. Also, it just intuitively makes sense; if we refer to our answer for question 3, b) we can see that k=2 is the only model without any crazy overlaps of distributions.

Question 3, f)

To fully address this question I have used the standard Scikit-learn implementation side by side with my own implementation of the GMM algorithm. Both implementations have the same result, which gives me confidence by implementation is correct, I have included the code in the relevant section in the appendix.

Appendix of evidence:

Question 1, a)

```python
#Question 1, part a)
#Implement the mixture Poisson PMF function
def mixture_poisson_pmf(k, alpha, lambda_val):
    """
    Calculate the PMF of a mixture of two Poisson distributions

    Args:
        k: The value at which to evaluate the PMF (integer)
        alpha: Mixture parameter between 0 and 1
        lambda_val: Base lambda parameter for the Poisson distributions

    Returns:
        The probability P(X = k)
    """
    # First Poisson component with parameter alpha*Lambda
    p1 = np.exp(-alpha * lambda_val) * ((alpha * lambda_val)**k) / factorial(k)

    # Second Poisson component with parameter (1-alpha)*Lambda
    p2 = np.exp(-(1-alpha) * lambda_val) * (((1-alpha) * lambda_val)**k) / factorial(k)

    # Equal mixture (weight 0.5 for each component)
    return 0.5 * p1 + 0.5 * p2

#Alternative implementation using scipy's poisson PMF
def mixture_poisson_pmf_scipy(k, alpha, lambda_val):
    """
    Calculate the PMF using scipy's poisson distribution
    """
    p1 = poisson.pmf(k, alpha * lambda_val)
    p2 = poisson.pmf(k, (1-alpha) * lambda_val)
    return 0.5 * p1 + 0.5 * p2

# Example calculation with specific values
alpha_example = 0.3
lambda_example = 5
k_example = 3

# Calculate using both methods to verify they give the same result
result_numpy = mixture_poisson_pmf(k_example, alpha_example, lambda_example)
result_scipy = mixture_poisson_pmf_scipy(k_example, alpha_example, lambda_example)

print(f"For k={k_example}, alpha={alpha_example}, lambda={lambda_example}:")
print(f"P(X = {k_example}) [numpy] = {result_numpy:.6f}")
print(f"P(X = {k_example}) [scipy] = {result_scipy:.6f}")

# Plot the PMF for a range of k values
k_range = range(20)
pmf_values = [mixture_poisson_pmf_scipy(k, alpha_example, lambda_example) for k in k_range]

plt.figure(figsize=(10, 6))
plt.bar(k_range, pmf_values, alpha=0.7)
plt.xlabel('k')
plt.ylabel('P(X = k)')
plt.title(f'PMF of Mixture of Two Poisson Distributions\nα={alpha_example}, λ={lambda_example}')
plt.grid(alpha=0.3)
plt.show()
```

Question 1, b)

```python
#question 1, part b)
def expected_value_mixture(alpha, lambda_val):
    """
    Calculate the expected value of the mixture distribution

    Args:
        alpha: Mixture parameter between 0 and 1
        lambda_val: Base lambda parameter for the Poisson distributions

    Returns:
        The expected value E[X]
    """
    # Theoretical expected value
    theoretical_E = lambda_val / 2

    # Numerical verification by sampling
    # Sample from first Poisson
    X1 = poisson.rvs(alpha * lambda_val, size=100000)
    # Sample from second Poisson
    X2 = poisson.rvs((1-alpha) * lambda_val, size=100000)
    # Combine samples with equal weights
    X = np.concatenate([X1, X2])
    # Calculate empirical mean
    empirical_E = np.mean(X)

    return theoretical_E, empirical_E

# Example calculation
alpha_example = 0.3
lambda_example = 5

theoretical, empirical = expected_value_mixture(alpha_example, lambda_example)

print(f"For alpha={alpha_example}, lambda={lambda_example}:")
print(f"Theoretical E[X] = {theoretical:.4f}")
print(f"Empirical E[X] = {empirical:.4f}")

# Plot the distribution and mark the expected value
k_range = range(20)
pmf_values = [0.5 * poisson.pmf(k, alpha_example * lambda_example) +
              0.5 * poisson.pmf(k, (1-alpha_example) * lambda_example)
              for k in k_range]

plt.figure(figsize=(10, 6))
plt.bar(k_range, pmf_values, alpha=0.7)
plt.axvline(x=theoretical, color='red', linestyle='--', label=f'E[X] = {theoretical:.2f}')
plt.xlabel('k')
plt.ylabel('P(X = k)')
plt.title(f'PMF of Mixture of Two Poisson Distributions\nα={alpha_example}, λ={lambda_example}')
plt.grid(alpha=0.3)
plt.legend()
plt.show()
```

Question 1, c)

```python
#question 1, part c)
def variance_mixture(alpha, lambda_val):
    """
    Calculate the variance of the mixture distribution

    Args:
        alpha: Mixture parameter between 0 and 1
        lambda_val: Base lambda parameter for the Poisson distributions

    Returns:
        The theoretical and empirical variance
    """
    # Theoretical variance calculation
    # For a Poisson distribution, variance equals mean
    var_X1 = alpha * lambda_val      # Variance of first Poisson
    var_X2 = (1-alpha) * lambda_val  # Variance of second Poisson

    # Law of total variance
    # E[Var(X|Z)] = 0.5*var_X1 + 0.5*var_X2 = lambda/2
    expected_var = 0.5 * var_X1 + 0.5 * var_X2

    # Var(E[X|Z]) = E[(E[X|Z])^2] - (E[E[X|Z]])^2
    mean_X1 = alpha * lambda_val
    mean_X2 = (1-alpha) * lambda_val
    var_means = 0.5 * (mean_X1**2 + mean_X2**2) - (0.5 * mean_X1 + 0.5 * mean_X2)**2

    theoretical_var = expected_var + var_means

    # Numerical verification by sampling
    # Sample from first Poisson
    X1 = poisson.rvs(alpha * lambda_val, size=100000)
    # Sample from second Poisson
    X2 = poisson.rvs((1-alpha) * lambda_val, size=100000)
    # Combine samples with equal weights
    X = np.concatenate([X1, X2])
    # Calculate empirical variance
    empirical_var = np.var(X)

    return theoretical_var, empirical_var

# Example calculation
alpha_example = 0.3
lambda_example = 5

theoretical_var, empirical_var = variance_mixture(alpha_example, lambda_example)

print(f"For alpha={alpha_example}, lambda={lambda_example}:")
print(f"Theoretical Var(X) = {theoretical_var:.4f}")
print(f"Empirical Var(X) = {empirical_var:.4f}")

# Plot the distribution and mark ±1 standard deviation
k_range = range(20)
pmf_values = [0.5 * poisson.pmf(k, alpha_example * lambda_example) +
              0.5 * poisson.pmf(k, (1-alpha_example) * lambda_example)
              for k in k_range]

mean = lambda_example / 2  # From part b)
std_dev = np.sqrt(theoretical_var)

plt.figure(figsize=(10, 6))
plt.bar(k_range, pmf_values, alpha=0.7)
plt.axvline(x=mean, color='red', linestyle='--', label=f'Mean = {mean:.2f}')
plt.axvline(x=mean - std_dev, color='green', linestyle=':', label=f'±1 std dev')
plt.axvline(x=mean + std_dev, color='green', linestyle=':')
plt.xlabel('k')
plt.ylabel('P(X = k)')
plt.title(f'PMF of Mixture of Two Poisson Distributions\nα={alpha_example}, λ={lambda_example}')
plt.grid(alpha=0.3)
plt.legend()
plt.show()
```

Question 2, a)

```python
#Question 2, part a)
# Load the data
london_data = np.loadtxt('London.csv', dtype=int)
ny_data = np.loadtxt('NY.csv', dtype=int)
doha_data = np.loadtxt('Doha.csv', dtype=int)
paris_data = np.loadtxt('Paris.csv', dtype=int)

# Create a figure with subplots
fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(12, 10))

# Plot normalized histograms
ax1.hist([london_data, ny_data, doha_data, paris_data],
         bins=range(0, 18),
         density=True,  # This normalizes the histograms
         label=['London', 'New York', 'Doha', 'Paris'],
         alpha=0.7)
ax1.set_xlabel('Number of Sales per Day')
ax1.set_ylabel('Relative Frequency')
ax1.set_title('Normalized Distribution of Daily Sales Across Dealerships')
ax1.legend()
ax1.grid(True, alpha=0.3)

# Plot box plots
ax2.boxplot([london_data, ny_data, doha_data, paris_data],
            labels=['London', 'New York', 'Doha', 'Paris'],
            showfliers=True)
ax2.set_ylabel('Number of Sales per Day')
ax2.set_title('Box Plot of Daily Sales Across Dealerships')
ax2.grid(True, alpha=0.3)

# Adjust layout and display
plt.tight_layout()
plt.show()

# Create summary statistics table
print("\nSummary Statistics:")
print("Dealership | Mean | Median | Std Dev | Min | Max | Sample Size")
print("-" * 60)
for name, data in [('London', london_data),
                   ('New York', ny_data),
                   ('Doha', doha_data),
                   ('Paris', paris_data)]:
    print(f"{name:10} | {np.mean(data):.2f} | {np.median(data):.2f} | "
          f"{np.std(data):.2f} | {np.min(data)} | {np.max(data)} | {len(data)}")
```

Question 2, b)

```python
#Question 2, part b)

def normalize_data(data):
    """Normalize data to relative frequencies"""
    unique, counts = np.unique(data, return_counts=True)
    total = len(data)
    return unique, counts/total

def poisson_log_likelihood(lambda_val, data):
    """Calculate negative log-likelihood for Poisson distribution"""
    k, p = data
    return -np.sum(p * poisson.logpmf(k, lambda_val))

def fit_poisson(data):
    """Fit Poisson distribution using MLE on normalized data"""
    k, p = normalize_data(data)
    lambda_init = np.mean(data)
    result = minimize(poisson_log_likelihood,
                      lambda_init,
                      args=((k, p),),
                      bounds=[(0.1, None)])
    return result.x[0]

def calculate_kl_divergence(p, q):
    """Calculate KL divergence between two probability distributions"""
    # Add small epsilon to avoid log(0)
    epsilon = 1e-10
    return np.sum(p * np.log((p + epsilon) / (q + epsilon)))

def compare_distributions(london_data, other_data, other_name):
    """Compare two dealerships using MLE and KL divergence"""
    # Fit Poisson distributions
    london_lambda = fit_poisson(london_data)
    other_lambda = fit_poisson(other_data)

    # Calculate KL divergence
    london_k, london_p = normalize_data(london_data)
    other_k, other_p = normalize_data(other_data)

    # Align the distributions to same k values
    all_k = np.unique(np.concatenate([london_k, other_k]))
    london_p_aligned = np.zeros_like(all_k, dtype=float)
    other_p_aligned = np.zeros_like(all_k, dtype=float)

    for i, k in enumerate(all_k):
        if k in london_k:
            london_p_aligned[i] = london_p[np.where(london_k == k)[0][0]]
        if k in other_k:
            other_p_aligned[i] = other_p[np.where(other_k == k)[0][0]]

    kl_div = calculate_kl_divergence(london_p_aligned, other_p_aligned)

    return {
        'name': other_name,
        'lambda': other_lambda,
        'lambda_diff': abs(london_lambda - other_lambda),
        'kl_divergence': kl_div
    }
```

```python
# Load data
london_data = np.loadtxt('London.csv')
ny_data = np.loadtxt('NY.csv')
doha_data = np.loadtxt('Doha.csv')
paris_data = np.loadtxt('Paris.csv')

# Compare each dealership with London
dealerships = {
    'New York': ny_data,
    'Doha': doha_data,
    'Paris': paris_data
}

print("=== Maximum Likelihood Analysis of Sales Patterns ===")
print("\nLondon's Sales Pattern:")
print("-" * 50)
london_lambda = fit_poisson(london_data)
print(f"Poisson λ: {london_lambda:.2f}")

print("\nComparison with Other Dealerships:")
print("-" * 50)

results = []
for name, data in dealerships.items():
    result = compare_distributions(london_data, data, name)
    results.append(result)
    print(f"\n{name}:")
    print(f"Poisson λ: {result['lambda']:.2f}")
    print(f"λ difference: {result['lambda_diff']:.2f}")
    print(f"KL Divergence: {result['kl_divergence']:.4f}")

# Find most similar dealership based on KL divergence
most_similar = min(results, key=lambda x: x['kl_divergence'])

print("\n" + "="*50)
print(f"Conclusion: {most_similar['name']} is most similar to London")
print("Based on:")
print(f"1. KL Divergence: {most_similar['kl_divergence']:.4f}")
print(f"2. λ difference: {most_similar['lambda_diff']:.2f}")
print("="*50)
```

Question 2, c)

```python
#question 2, part c)

def poisson_log_likelihood(lambda_val, data):
    """Calculate negative log-likelihood for Poisson distribution"""
    return -np.sum(poisson.logpmf(data, lambda_val))

def neg_bin_log_likelihood(params, data):
    """Calculate negative log-likelihood for Negative Binomial distribution"""
    r, p = params
    # Add small epsilon to avoid log(0)
    epsilon = 1e-10
    return -np.sum(np.log(nbinom.pmf(data, r, p) + epsilon))

def binomial_log_likelihood(params, data):
    """Calculate negative log-likelihood for Binomial distribution"""
    n, p = params
    n = int(n)  # Ensure n is integer
    # Add small epsilon to avoid log(0)
    epsilon = 1e-10
    return -np.sum(np.log(binom.pmf(data, n, p) + epsilon))

def fit_distribution(data, distribution_type):
    """Fit distribution to data using MLE"""
    try:
        if distribution_type == 'poisson':
            lambda_init = np.mean(data)
            result = minimize(poisson_log_likelihood,
                              lambda_init,
                              args=(data,),
                              bounds=[(0.1, None)])
            return {
                'params': [result.x[0]],
                'log_likelihood': -result.fun,
                'aic': 2 * 1 - 2 * (-result.fun)
            }

        elif distribution_type == 'negative_binomial':
            mean = np.mean(data)
            var = np.var(data)
            # Better initial parameter estimates
            p_init = min(0.99, mean / var)
            r_init = max(0.1, mean * p_init / (1 - p_init))

            result = minimize(neg_bin_log_likelihood,
                              [r_init, p_init],
                              args=(data,),
                              bounds=[(0.1, None), (0.01, 0.99)])
            return {
                'params': result.x,
                'log_likelihood': -result.fun,
                'aic': 2 * 2 - 2 * (-result.fun)
            }

        elif distribution_type == 'binomial':
            n_init = int(np.max(data) * 2)  # Better initial guess for n
            p_init = min(0.99, np.mean(data) / n_init)

            result = minimize(binomial_log_likelihood,
                              [n_init, p_init],
                              args=(data,),
                              bounds=[(np.max(data), None), (0.01, 0.99)])
            return {
                'params': result.x,
                'log_likelihood': -result.fun,
                'aic': 2 * 2 - 2 * (-result.fun)
            }
    except Exception as e:
        print(f"Error fitting {distribution_type}: {str(e)}")
        return {
            'params': [np.nan],
            'log_likelihood': np.nan,
            'aic': np.nan
        }

# Load data
london_data = np.loadtxt('London.csv')
ny_data = np.loadtxt('NY.csv')
doha_data = np.loadtxt('Doha.csv')
paris_data = np.loadtxt('Paris.csv')

dealerships = {
    'London': london_data,
    'New York': ny_data,
    'Doha': doha_data,
    'Paris': paris_data
}

distributions = ['poisson', 'negative_binomial', 'binomial']

print("=== Distribution Fitting Analysis ===")
print("="*50)

# Suppress optimization warnings
warnings.filterwarnings('ignore', category=RuntimeWarning)

for name, data in dealerships.items():
    print(f"\n{name} Analysis:")
    print("-"*30)

    results = {}
    for dist in distributions:
        results[dist] = fit_distribution(data, dist)

        print(f"\n{dist.title()} Distribution:")
        if not np.isnan(results[dist]['log_likelihood']):
            print(f"Parameters: {[f'{p:.2f}' for p in results[dist]['params']]}")
            print(f"Log-likelihood: {results[dist]['log_likelihood']:.2f}")
            print(f"AIC: {results[dist]['aic']:.2f}")
        else:
            print("Failed to fit distribution")
```

```python
    # Find best fitting distribution (excluding failed fits)
    valid_results = {k: v for k, v in results.items() if not np.isnan(v['aic'])}
    if valid_results:
        best_dist = min(valid_results.items(), key=lambda x: x[1]['aic'])[0]
        print(f"\nBest fitting distribution: {best_dist.title()}")
    else:
        print("\nNo distribution could be successfully fitted")
    print("-"*30)
```

Question 3, My model + scikit-learn's implementation

```python
#question 3, part a)

# Load the data
# The file has three columns: index, eruptions, and waiting
data = pd.read_csv('OldFaithful.dat', sep=r'\s+', header=0,
                   names=['index', 'eruptions', 'waiting'])

# Get the starting point using last two digits of student number
start_serial = 90  # Last two digits of 240355490

# Find the index of the row with serial number 90
start_index = data[data['index'] == start_serial].index[0]

# Select the next 150 points
selected_data = data.iloc[start_index:start_index + 150]

# Print information about the selected data
print(f"Starting from serial number: {start_serial}")
print(f"Selected data shape: {selected_data.shape}")
print("\nFirst few rows of selected data:")
print(selected_data.head())

# Prepare the data for GMM (we'll use both eruption and waiting time)
X = selected_data[['eruptions', 'waiting']].values

# Now we can proceed with the GMM implementation
class GMM_EM:
    def __init__(self, n_components, max_iter=100, tol=1e-6):
        self.n_components = n_components
        self.max_iter = max_iter
        self.tol = tol

    def fit(self, X):
        n_samples, n_features = X.shape

        # Initialize parameters
        self.weights = np.ones(self.n_components) / self.n_components
        self.means = X[np.random.choice(n_samples, self.n_components, replace=False)]
        self.covariances = [np.eye(n_features) for _ in range(self.n_components)]

        # EM algorithm
        log_likelihood_old = -np.inf
        log_likelihood_history = []

        for iteration in range(self.max_iter):
            # E-step: Calculate responsibilities
            responsibilities = np.zeros((n_samples, self.n_components))
            for k in range(self.n_components):
                responsibilities[:, k] = self.weights[k] * multivariate_normal.pdf(
                    X, mean=self.means[k], cov=self.covariances[k])

            # Normalize responsibilities
            responsibilities = responsibilities / responsibilities.sum(axis=1, keepdims=True)
```

```
            # Normalize responsibilities
            responsibilities = responsibilities / responsibilities.sum(axis=1, keepdims=True)

            # M-step: Update parameters
            Nk = responsibilities.sum(axis=0)

            # Update weights
            self.weights = Nk / n_samples

            # Update means
            for k in range(self.n_components):
                self.means[k] = np.sum(responsibilities[:, k:k+1] * X, axis=0) / Nk[k]

                # Update covariances
                diff = X - self.means[k]
                self.covariances[k] = np.dot(responsibilities[:, k] * diff.T, diff) / Nk[k]

            # Calculate log-likelihood
            log_likelihood = 0
            for k in range(self.n_components):
                log_likelihood += self.weights[k] * multivariate_normal.pdf(
                    X, mean=self.means[k], cov=self.covariances[k])
            log_likelihood = np.sum(np.log(log_likelihood))
            log_likelihood_history.append(log_likelihood)

            # Check convergence
            if abs(log_likelihood - log_likelihood_old) < self.tol:
                break

            log_likelihood_old = log_likelihood

        return log_likelihood_history

    def predict(self, X):
        responsibilities = np.zeros((X.shape[0], self.n_components))
        for k in range(self.n_components):
            responsibilities[:, k] = self.weights[k] * multivariate_normal.pdf(
                X, mean=self.means[k], cov=self.covariances[k])
        return responsibilities.argmax(axis=1)
```

```python
# Using scikit-learn's implementation
gmm_sklearn = GaussianMixture(n_components=2, random_state=42)
gmm_sklearn.fit(X)

# Using our implementation
gmm_custom = GMM_EM(n_components=2)
log_likelihood_history = gmm_custom.fit(X)

# Plot the results
def plot_gmm_results(X, gmm, title, is_sklearn=False):
    plt.figure(figsize=(10, 6))

    # Plot the data points
    plt.scatter(X[:, 0], X[:, 1], alpha=0.5)

    # Plot the Gaussian components
    for k in range(gmm.n_components):
        # Create grid for contour plot
        x = np.linspace(X[:, 0].min() - 1, X[:, 0].max() + 1, 100)
        y = np.linspace(X[:, 1].min() - 1, X[:, 1].max() + 1, 100)
        X_grid, Y_grid = np.meshgrid(x, y)
        pos = np.dstack((X_grid, Y_grid))

        # Calculate PDF
        if is_sklearn:
            rv = multivariate_normal(gmm.means_[k], gmm.covariances_[k])
        else:
            rv = multivariate_normal(gmm.means[k], gmm.covariances[k])
        Z = rv.pdf(pos)

        # Plot contour
        plt.contour(X_grid, Y_grid, Z, levels=3, colors='red', alpha=0.5)

    plt.title(title)
    plt.xlabel('Eruption Time (minutes)')
    plt.ylabel('Waiting Time (minutes)')
    plt.show()

# Plot results from both implementations
plot_gmm_results(X, gmm_custom, 'Custom GMM Implementation', is_sklearn=False)
plot_gmm_results(X, gmm_sklearn, 'Scikit-learn GMM Implementation', is_sklearn=True)

# Plot log-likelihood progression
plt.figure(figsize=(10, 6))
plt.plot(log_likelihood_history)
plt.title('Log-likelihood Progression')
plt.xlabel('Iteration')
plt.ylabel('Log-likelihood')
plt.grid(True)
plt.show()
```

Question 3, b)

```python
#question 3, b)

# Load the data
data = pd.read_csv('OldFaithful.dat', sep=r'\s+', header=0,
                   names=['index', 'eruptions', 'waiting'])

# Get the starting point using last two digits of student number
start_serial = 90  # Last two digits of 24035490
start_index = data[data['index'] == start_serial].index[0]

# Select the next 150 points
selected_data = data.iloc[start_index:start_index + 150]

# Prepare the data for GMM
X = selected_data[['eruptions', 'waiting']].values

# Define the number of components to try
n_components_list = [2, 3, 4, 5]

# Create a figure with subplots
fig, axes = plt.subplots(2, 2, figsize=(15, 12))
axes = axes.ravel()

# Function to plot GMM results
def plot_gmm_results(X, gmm, ax, title):
    # Plot the data points
    ax.scatter(X[:, 0], X[:, 1], alpha=0.5, s=10)

    # Plot the Gaussian components
    for k in range(gmm.n_components):
        # Create grid for contour plot
        x = np.linspace(X[:, 0].min() - 1, X[:, 0].max() + 1, 100)
        y = np.linspace(X[:, 1].min() - 1, X[:, 1].max() + 1, 100)
        X_grid, Y_grid = np.meshgrid(x, y)
        pos = np.dstack((X_grid, Y_grid))

        # Calculate PDF
        rv = multivariate_normal(gmm.means_[k], gmm.covariances_[k])
        Z = rv.pdf(pos)

        # Plot contour
        ax.contour(X_grid, Y_grid, Z, levels=3, colors='red', alpha=0.5)

        # Plot the mean
        ax.scatter(gmm.means_[k][0], gmm.means_[k][1],
                   marker='x', color='red', s=100, linewidth=2)

    ax.set_title(title)
    ax.set_xlabel('Eruption Time (minutes)')
    ax.set_ylabel('Waiting Time (minutes)')
    ax.grid(True, alpha=0.3)

# Train and plot GMMs for different numbers of components
for i, n_components in enumerate(n_components_list):
    # Train GMM
    gmm = GaussianMixture(n_components=n_components, random_state=42)
    gmm.fit(X)

    # Plot results
    plot_gmm_results(X, gmm, axes[i],
                     f'GMM with {n_components} Components\nBIC: {gmm.bic(X):.2f}')

plt.tight_layout()
plt.show()

# Print BIC scores for model comparison
print("\nBIC scores for different numbers of components:")
for n_components in n_components_list:
    gmm = GaussianMixture(n_components=n_components, random_state=42)
    gmm.fit(X)
    print(f"K={n_components}: BIC = {gmm.bic(X):.2f}")
```

Question 3, c)

```python
#Question 3, c)

# Load the data
data = pd.read_csv('OldFaithful.dat', sep=r'\s+', header=0,
                   names=['index', 'eruptions', 'waiting'])

# Get the starting point using last two digits of student number
start_serial = 90  # Last two digits of 240355490
start_index = data[data['index'] == start_serial].index[0]

# Select the next 150 points
selected_data = data.iloc[start_index:start_index + 150]

# Prepare the data for GMM
X = selected_data[['eruptions', 'waiting']].values

# Define the number of components to try
n_components_list = [2, 3, 4, 5]

# Create a figure for likelihood progression
plt.figure(figsize=(15, 10))

# Train GMMs and plot likelihood progression for each number of components
for n_components in n_components_list:
    # Create GMM with verbose=2 to get iteration information
    gmm = GaussianMixture(n_components=n_components,
                          random_state=42,
                          verbose=2,  # This will print iteration info
                          verbose_interval=1)  # Print every iteration

    # Fit the model
    gmm.fit(X)

    # Get the likelihood values from the verbose output
    # Note: This is a workaround since scikit-learn doesn't directly provide iteration history
    # We'll use the log-likelihood values from the model's score_samples method
    # and calculate the progression manually

    # Calculate likelihood progression
    likelihood_progression = []
    for i in range(1, gmm.n_iter_ + 1):
        # Create a temporary model with the same parameters
        temp_gmm = GaussianMixture(n_components=n_components,
                                   random_state=42,
                                   max_iter=i)
        temp_gmm.fit(X)
        # Calculate total log-likelihood
        log_likelihood = temp_gmm.score(X) * len(X)
        likelihood_progression.append(log_likelihood)

    # Plot the likelihood progression
    plt.plot(range(1, len(likelihood_progression) + 1),
             likelihood_progression,
             label=f'K={n_components}',
             marker='o',
             markersize=4)

plt.title('Log-Likelihood Progression During EM Algorithm')
plt.xlabel('Iteration')
plt.ylabel('Log-Likelihood')
plt.grid(True, alpha=0.3)
plt.legend()
plt.show()

# Print final likelihood values and number of iterations
print("\nFinal results for each model:")
for n_components in n_components_list:
    gmm = GaussianMixture(n_components=n_components, random_state=42)
    gmm.fit(X)
    print(f"K={n_components}:")
    print(f"  Final log-likelihood: {gmm.score(X) * len(X):.2f}")
    print(f"  Number of iterations: {gmm.n_iter_}")
```